

CAPSTONE PROJECT REPORT

SAP BTP AI Development

JouleOps @ NorthWind Manufacturing

Enterprise Operations Assistant using

SAP HANA Cloud, FastAPI, MCP and SAP BTP

Submitted by : Susmitha Boga

Date: August 17, 2026

Platform : SAP BTP Trial Account

Table of Contents

1. Introduction and Purpose
2. Problem Statement
3. Component Responsibilities
4. MCP Interface
5. BTP Destination Setup
6. Local Runtime Sequence
7. Architecture Summary
8. Complete Project Step by Step Process
9. Final Execution Flow
10. What to run in vscode
11. Output Screenshots
12. Conclusion

1. Introduction and Purpose

JouleOps is the enterprise operations assistant built for the NorthWind Manufacturing capstone. The main purpose of the project is to connect business data in SAP HANA Cloud with a controlled AI-facing layer.

I built the application in separate layers so that each part can be tested independently. SAP HANA Cloud stores the data, FastAPI provides the business APIs, MCP exposes selected capabilities as tools, ngrok gives temporary HTTPS access during development, and SAP BTP Destinations connect the SAP side to the backend.

2. Problem Statement

In a manufacturing environment, information is usually divided across different business areas. A person may need to check material stock, open sales orders, customer exposure, invoices and maintenance tickets separately.

The problem this project solves is to provide one controlled entry point for these operations. Instead of allowing an AI system to directly access the HANA database, the AI uses defined business capabilities.

Business Need	Application Response
Check material stock	get_material_details
Find open orders	get_open_sales_orders
Understand customer exposure	get_customer_summary
Create maintenance ticket	create_ticket
Understand overdue invoices	summarize_overdue_invoices

Note :

The AI should call a business function, not write arbitrary SQL. This makes the application easier to control, test and audit.

3. Component Responsibilities

Component	Simple Explanation
Joule Agent	Interprets user intent and selects the appropriate tool.
SAP HANA Cloud	Database for NorthWind business information
FastAPI	Validate requests, executes business logic and accesses HANA.
MCP Server	exposes selected capabilities through MCP.
ngrok	Temporary public HTTPS tunnel during development
BTP Destination	Stores the endpoint configuration for SAP BTP
Joule/Skills	AI-facing layer that chooses a suitable business capability

4. MCP Interface

- `get_inventory_snapshot(plant_code)`: plant-level inventory summary.
- `get_overdue_invoices(region, days_overdue_min)`: filtered overdue invoice information.
- `create_ticket(...)`: same governed write action available through the MCP path.

5. BTP Destination Setup

- `JouleOps_FastAPI`: HTTP destination pointing to the FastAPI public endpoint.
- Set `sap-joule-studio-action = true`.
- `JouleOps_MCP`: HTTP destination pointing to the MCP public endpoint.
- Set `sap-joule-studio-mcp-server = true`.
- For the documented ngrok trial configuration, use Internet proxy and NoAuthentication.

Run Check Connection on both destinations.

6. Local Runtime Sequence

1. Activate `.venv`.
2. Start FastAPI with Uvicorn on port 8000.
3. Open Swagger UI and test the REST endpoints.
4. Start the MCP Server on its configured port.
5. Create the required ngrok HTTPS tunnel(s).
6. Copy the public URL into the corresponding BTP Destination.
7. Check both BTP Destination connections.
8. Proceed to Joule discovery/configuration only after connectivity succeeds.

7. Architecture Summary

The final governed path is: User → Joule Agent → Skill or MCP → BTP Destination → FastAPI/MCP → SAP HANA Cloud → JSON → Joule Agent → User.

8. Complete Project Step-by-Step Process

The following sequence follows the implementation order documented in the supplied capstone report. It separates infrastructure setup, backend development, connectivity and final Joule integration.

Step 1 — SAP BTP Trial Setup

Log in to the SAP BTP Trial account and open the trial subaccount. Verify the available services, entitlements and Cloud Foundry environment.

Step 2 — SAP HANA Cloud Setup

Open Services → Service Marketplace, select SAP HANA Cloud and provision the required trial/free-tier instance. Confirm the instance under Instances and Subscriptions.

Credential handling

The HANA password and service-key contents should stay outside the report. They belong in secure local configuration such as environment variables.

Step 3 — VS Code and Python Environment

Create the project in Visual Studio Code, create the .venv environment and install the required Python packages such as FastAPI, Uvicorn, hdbcli, httpx and Pydantic.

```
python -m venv .venv
```

```
.\venv\Scripts\Activate.ps1
```

```
pip install fastapi uvicorn hdbcli httpx pydantic python-dotenv fastmcp
```

```
pip freeze > requirements.txt
```

Step 4 — Database Schema and Seed Data

Create the six NorthWind tables: MATERIALS, SALES_ORDERS, CUSTOMERS, INVOICES, TICKETS and AUDIT_LOG. Run the Python data-generation script and verify the inserted records in HANA Database Explorer.

Step 5 — FastAPI Database Layer

Create the HANA connection layer using hdbcli. Keep HANA credentials in backend environment variables. Use SQL parameter binding and validation rather than string concatenation.

Step 6 — Implement the Five FastAPI Capabilities

Implement get_material_details, get_open_sales_orders, get_customer_summary, create_ticket and summarize_overdue_invoices. The create_ticket write operation must also create the required AUDIT_LOG record.

Step 7 — Start and Test FastAPI

Run the FastAPI application with Uvicorn on port 8000. Open Swagger UI and test all five capabilities. Confirm that the responses contain data retrieved from HANA Cloud.

Step 8 — Implement the MCP Server

Create the custom MCP server and expose the selected tools: get_inventory_snapshot, get_overdue_invoices and create_ticket. Run the MCP server alongside the FastAPI backend.

Step 9 — Create the Public HTTPS Tunnel

Run ngrok against the locally running backend service. The supplied report documents the FastAPI service running on port 8000 and ngrok forwarding public HTTPS traffic to localhost:8000.

Step 10 — Configure BTP Destinations

In BTP Cockpit open Connectivity → Destinations. Create JouleOps_FastAPI with the FastAPI/ngrok URL and sap-joule-studio-action = true. Create JouleOps_MCP for the MCP endpoint and set sap-joule-studio-

mcp-server = true. The report documents Internet proxy and NoAuthentication for the ngrok trial configuration.

Step 11 — Verify Destination Connectivity

Use Check Connection for both destinations. A successful connection proves that SAP BTP can reach the local/public backend path through the configured tunnel.

Step 12 — Joule Skill and MCP Integration

When the SAP Build/Joule Studio UI is available, register the five REST actions as Joule Skills and allow the MCP destination to expose the selected MCP tools. The agent then chooses the appropriate path based on intent.

Step 13 — Execute the Five Demo Scenarios

Validate the stock-check/ticket scenario, open-sales-orders scenario, customer-exposure scenario, MCP inventory scenario and the ticket/RBAC guardrail scenario.

Step 14 — Final Validation

Verify HANA data, Swagger responses, MCP startup, destination connectivity, ticket creation and matching audit records. The supplied report states that the shared trial tenant restricted live Joule UI access, so the backend architecture was validated independently.

9. Final Execution Flow

BTP Trial Setup → HANA Cloud → Database Tables/Data → FastAPI → Swagger Validation → MCP Server → ngrok → BTP Destinations → Joule Skills/MCP Discovery → Joule Agent → Demo Scenarios → Validation

10. What to Run in VS Code

- Terminal 1: start the FastAPI application with Uvicorn on port 8000.
- Terminal 2: start the MCP Server on its configured port.
- Terminal 3: start ngrok for the public endpoint required by the BTP Destination.
- Keep the application, MCP server and tunnel running while testing BTP connectivity.

11. Output Screenshots

11.1 HANA Cloud Tables

Show JOULEOPS schema and the six tables: MATERIALS, CUSTOMERS, SALES_ORDERS, INVOICES, TICKETS and AUDIT_LOG.}]

SAP HANA Database Explorer

HC_HDB (DBADMIN): DBADMIN.MATERIALS

Filter Instances

- Synonyms
- Table Types
- Tables
- Tasks
- Triggers
- Views
- Database Diagnostic Files
- HDI Containers

DBADMIN

Search Tables

AUDIT_LOG

CUSTOMERS

INVOICES

MATERIALS

SALES_ORDERS

Table Name: MATERIALS Schema: DBADMIN

Columns

	Name	SQL Data Type	Key	Not Null	Default	Comment
1	MATERIAL_ID	NVARCHAR(20)	1	X	NULL	
2	DESCRIPTION	NVARCHAR(255)		X	NULL	
3	CATEGORY	NVARCHAR(50)		X	NULL	
4	UNIT_PRICE	DECIMAL(15,2)		X	NULL	
5	STOCK_QTY	INTEGER		X	NULL	
6	SAFETY_STOCK	INTEGER		X	NULL	
7	PLANT_CODE	NVARCHAR(10)		X	NULL	
8	CREATED_ON	TIMESTAMP			CURRENT_T	
9	UPDATED_ON	TIMESTAMP			CURRENT_T	
10	SOURCE_SYSTEM	NVARCHAR(50)			HANA_CLOU	

Activate Windows
Go to Settings to activate Windows.

11.2 HANA Demo Data

SAP HANA Database Explorer

HC_HDB (DBADMIN): DBADMIN.CUSTOMERS

Filter Instances

- Synonyms
- Table Types
- Tables
- Tasks
- Triggers
- Views
- Database Diagnostic Files
- HDI Containers

DBADMIN

Search Tables

AUDIT_LOG

CUSTOMERS

INVOICES

MATERIALS

SALES_ORDERS

Raw Data

Rows (100)

	CUSTOMER_ID	NAME	REGION	CREDIT_LIMIT	OUTSTANDING_AMOUNT	
1	C-001	Campos, Romero and Wilson	South	151190.29	20773.95	2025-03-
2	C-002	Meyer, Valenzuela and Mann	East	32949.84	15319.55	2025-06-
3	C-003	Hamilton-Caldwell	Central	195444.72	15707.30	2026-07-
4	C-004	Huynh, Johnson and Torres	Central	91241.46	41462.76	2024-10-
5	C-005	Myers-Kennedy	East	196152.69	3779.95	2024-12-
6	C-006	Woods Inc	South	89081.35	22066.77	2026-02-
7	C-007	Nguyen, Mitchell and Choi	East	122120.53	16796.19	2026-08-
8	C-008	Ewing, Jacobs and May	Central	198964.05	43153.56	2025-07-
9	C-009	Whitaker, Miller and Blankenship	South	100871.89	2736.66	2025-02-
10	C-010	Smith, Walton and Flores	South	62503.14	41846.24	2025-03-
11	C-011	Cooper Ltd	South	156866.61	42864.53	2025-02-
12	C-012	Jones-Gutierrez	East	12989.64	7188.14	2024-08-
13	C-013	Mccoy-Price	East	190898.13	26670.35	2025-05-
14	C-014	Bowen, Green and Gray	West	140323.08	25094.46	2026-07-
15	C-015	James-Reyes	West	75060.00	38610.75	2025-06-
16	C-016	Sexton Ltd	North	32561.72	44174.03	2025-05-
17	C-017	Fsnarza, Medina and Atkinson	South	171600.28	37198.80	2026-07-

Activate Windows
Go to Settings to activate Windows.

Type here to search

29°C Partly sunny

12:24
15-08-2026

11.3 Swagger Endpoint List

JouleOps API 0.1.0 OAS 3.1
/openapi.json

default

GET	/health	Health	▼
POST	/api/material/details	Get Material	▼
POST	/api/sales/orders	Get Sales	▼
POST	/api/customer/summary	Get Customer	▼
POST	/api/ticket/create	Create Ticket	▼
POST	/api/invoices/overdue	Overdue Invoices	▼

Schemas

CustomerRequest > Expand all object

HTTPValidationError > Expand all object

Activate Windows
Go to Settings to activate Windows.

GET /health Health

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/health' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/health
```

Server response

Code

Details

200

Response body

```
{
  "status": "ok"
}
```

Response headers

```
content-length: 15
content-type: application/json
date: Sat, 15 Aug 2026 07:01:19 GMT
server: uvicorn
```

Cancel

Download

Activate Windows
Go to Settings to activate Windows.

11.4 Material API Output

The image displays two screenshots of a REST client interface, likely Swagger UI or Postman, showing API calls and their request bodies.

Top Screenshot: POST /api/material/details

- Method:** POST
- Endpoint:** /api/material/details
- Parameters:** No parameters
- Request body:** application/json
- Request Body (JSON):**

```
{  "material_id": "MAT-0001",  "plant_code": "PLT-CHE"}
```
- Buttons:** Execute, Clear
- Responses:** (Empty)
- Curl Command:**

```
curl -X 'POST' \  'http://127.0.0.1:8000/api/material/details' \  -H 'Content-Type: application/json'
```

Bottom Screenshot: POST /api/sales/orders

- Method:** POST
- Endpoint:** /api/sales/orders
- Parameters:** No parameters
- Request body:** application/json
- Request Body (JSON):**

```
{  "region": "East",  "date_from": "2026-08-13",  "date_to": "2026-08-13"}
```
- Buttons:** Execute, Clear
- Responses:** (Empty)
- Curl Command:**

```
curl -X 'POST' \  'http://127.0.0.1:8000/api/sales/orders' \  -H 'Content-Type: application/json'
```

Both screenshots show a Windows taskbar at the bottom with the time 12:37 and 12:43, and a system tray indicating 29°C and Partly sunny weather.

127.0.0.1:8000/docs#/default/get_material_api_material_details_post

```
-d '{
  "material_id": "MAT-0001",
  "plant_code": "PLT-CHE"
}'
```

Request URL

http://127.0.0.1:8000/api/material/details

Server response

Code	Details
200	Response body <pre>{ "material_id": "MAT-0001", "description": "Distributed intangible hub", "category": "Finished Goods", "unit_price": 746.79, "stock_qty": 285, "safety_stock": 37, "plant_code": "PLT-CHE", "is_below_safety": false, "tool": "get_material_details", "source_tables": ["MATERIALS"] }</pre> Response headers <pre>content-length: 258 content-type: application/json date: Sat, 15 Aug 2026 07:07:28 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Controls Accept header.

Example Value | Schema

"string"

Activate Windows
Go to Settings to activate Windows.

127.0.0.1:8000/docs#/default/get_sales_api_sales_orders_post

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/sales/orders' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "region": "East",
    "date_from": "2026-08-13",
    "date_to": "2026-08-13"
  }'
```

Request URL

http://127.0.0.1:8000/api/sales/orders

Server response

Code	Details
200	Response body <pre>{ "region": "East", "orders": [], "tool": "get_open_sales_orders", "source_tables": ["SALES_ORDERS"] }</pre> Response headers <pre>content-length: 83 content-type: application/json date: Sat, 15 Aug 2026 07:12:58 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Controls Accept header.

Example Value | Schema

"string"

Activate Windows
Go to Settings to activate Windows.

127.0.0.1:8000/docs#/default/get_customer_api_customer_summary_post

POST /api/customer/summary Get Customer

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "customer_id": "C-003"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/customer/summary' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "customer_id": "C-003"
  }'
```

Request URL

http://127.0.0.1:8000/api/customer/summary

Server response

Code	Details
200	Response body <pre>{ "customer_id": "C-003", "name": "Hamilton-Caldwell", "region": "Central", "credit_limit": 15444.72, "outstanding_amount": 15707.3, "total_invoices": 1, "total_amount": 2304.11, "overdue_amount": 0, "tool": "get_customer_summary", "source_tables": ["CUSTOMERS", "INVOICES"] }</pre> Response headers <pre>content-length: 255 content-type: application/json date: Sat, 15 Aug 2026 07:17:04 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

11.7 Create Ticket Output

The image shows a web browser window displaying the output of a REST client. The address bar shows the URL `127.0.0.1:8000/docs#/default/create_ticket_api_ticket_create_post`. The page content is divided into sections: Curl, Request URL, Server response, and Responses.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/ticket/create' \
  -H 'accept: application/json' \
  -H 'x-user-role: PLANT_SUPERVISOR' \
  -H 'Content-Type: application/json' \
  -d '{
    "material_id": "MAT-1023",
    "plant_code": "PLT-PUN",
    "priority": "High",
    "assigned_team": "Mechanical",
    "description": "Material stock is below safety stock and requires inspection",
    "created_by_role": "PLANT_SUPERVISOR"
  }'
```

Request URL

`http://127.0.0.1:8000/api/ticket/create`

Server response

Code	Details
200	Response body <pre>{ "ticket_id": "TKT-20260816082242", "message": "Ticket TKT-20260816082242 created successfully", "assigned_team": "Mechanical", "priority": "High", "tool": "create ticket", "source_files": ["TICKETS", "AUDIT_LOG"] }</pre> Response headers <pre>content-length: 203 content-type: application/json date: Sun, 16 Aug 2026 02:52:39 GMT server: uvicorn</pre>

Responses

Code	Description
200	Successful Response

The bottom part of the image shows a REST client interface for the `POST /api/ticket/create` endpoint. The **Parameters** section shows `x-user-role` set to `PLANT_SUPERVISOR`. The **Request body** is set to `application/json`. The **Edit Value** section shows the JSON payload:

```
{
  "material_id": "MAT-1023",
  "plant_code": "PLT-PUN",
  "priority": "High",
  "assigned_team": "Mechanical",
  "description": "Material stock is below safety stock and requires inspection",
  "created_by_role": "PLANT_SUPERVISOR"
}
```

The **Execute** button is visible at the bottom of the interface.

11.8 Overdue Invoice Output

Code Details

200

Response body

```
{
  "customer_id": "C-068",
  "total_overdue": 40388.53,
  "max_days_overdue": 64,
  "recommendation": "HOLD_SHIPMENTS",
  "invoices": [
    {
      "invoice_id": "INV-0352",
      "amount": 40388.53,
      "due_date": "2026-06-10",
      "days_overdue": 64,
      "status": "Overdue"
    }
  ],
  "tool": "summarize_overdue_invoices",
  "source_tables": [
    "INVOICES"
  ]
}
```

Response headers

```
content-length: 287
content-type: application/json
date: Sun, 16 Aug 2026 03:15:01 GMT
server: unicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Media type: application/json

Controls: Accept header

Example Value | Schema

```
"string"
```

Windows taskbar: Type here to search, BSE midcap -0.13%, 12:01 16-08-2026

POST /api/invoices/overdue Overdue Invoices

Parameters

No parameters

Request body ^{required}

application/json

Edit Value | Schema

```
{
  "customer_id": "C-068"
}
```

Execute Clear

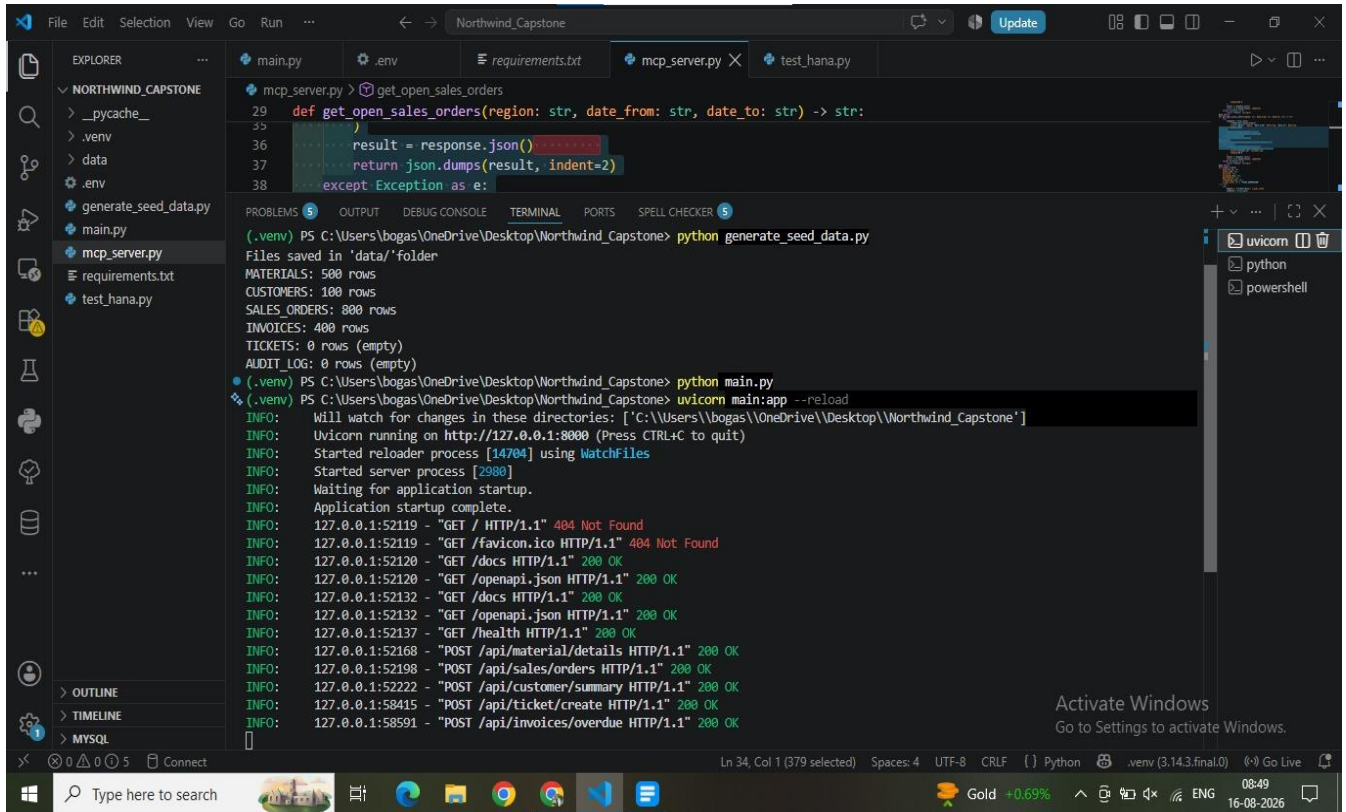
Responses

Curl

```
curl -X POST \
```

Windows taskbar: Type here to search, BSE midcap -0.13%, 12:01 16-08-2026

11.9 FastAPI Runtime



```
def get_open_sales_orders(region: str, date_from: str, date_to: str) -> str:
    ...
    result = response.json()
    return json.dumps(result, indent=2)
except Exception as e:
```

```
(.venv) PS C:\Users\bogas\OneDrive\Desktop\Northwind_Capstone> python generate_seed_data.py
Files saved in 'data/' folder
MATERIALS: 500 rows
CUSTOMERS: 100 rows
SALES_ORDERS: 800 rows
INVOICES: 400 rows
TICKETS: 0 rows (empty)
AUDIT_LOG: 0 rows (empty)

(.venv) PS C:\Users\bogas\OneDrive\Desktop\Northwind_Capstone> python main.py
(.venv) PS C:\Users\bogas\OneDrive\Desktop\Northwind_Capstone> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\bogas\OneDrive\Desktop\Northwind_Capstone']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [14704] using WatchFiles
INFO: Started server process [2080]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:52119 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52119 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52120 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:52120 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:52132 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:52132 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:52137 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:52168 - "POST /api/material/details HTTP/1.1" 200 OK
INFO: 127.0.0.1:52198 - "POST /api/sales/orders HTTP/1.1" 200 OK
INFO: 127.0.0.1:52222 - "POST /api/customer/summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:58415 - "POST /api/ticket/create HTTP/1.1" 200 OK
INFO: 127.0.0.1:58591 - "POST /api/invoices/overdue HTTP/1.1" 200 OK
```

11.10 MCP Runtime



```
(.venv) PS C:\Users\bogas\OneDrive\Desktop\Northwind_Capstone> python mcp_server.py
- summarize_overdue_invoices

=====
MCP Server is running!
Press Ctrl+C to stop
=====

INFO: Started server process [17132]
INFO: Waiting for application startup.
[08/15/26 12:28:17] INFO StreamableHTTP session manager started
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

11.11 ngrok

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 5
ngrok
One gateway for every AI model. https://app.ngrok.ai

Session Status      online
Account             susmithaboga373@gmail.com (Plan: Free)
Version             3.39.11-msix-stable
Region              India (in)
Latency              25ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://rumbling-goon-harmony.ngrok-free.dev -> http://localhost:8000

Connections          ttl    opn    rt1    rt5    p50    p90
                    0      0      0.00   0.00   0.00   0.00
```

11.13 BTP FastAPI Destination and BTP MCP Destination

Trial Home / 7b84e77ctrial / trial

Subaccount: trial - Destinations

Complete a 5-minute survey on the Destination service and help us improve your product experience! The survey will open in a new tab.

Search S... Q

Context Level Privile...

- trial - Destinations
 - trial Read and
 - Subscriptions (0)

Destinations

All: 2

Name	Type	Proxy Type	Labels	Description
JouleOps_FastAPI	HTTP	Internet	Add Label	>
JouleOps-MCP	HTTP	Internet	Add Label	>

Results per page: 50 1 2 Results

11.15 BTP Check Connection

The screenshot displays the SAP BTP Cockpit interface. On the left, a navigation menu includes 'Favorites & Recent', 'Subaccount', 'Overview', 'Services', 'Instances and Subscriptions', 'HTML5 Applications', 'Connectivity', 'Destinations', 'Destination Certificates', 'Destination Trust', 'Cloud Connectors', 'Security', 'Users', 'Role Collections', and 'Roles'. The 'Destinations' section is active, showing a table with two entries: 'JouleOps_Fa...' and 'JouleOps-MCP', both of type 'HTTP'. The 'JouleOps_FastAPI' destination is selected, and its details are shown on the right. A 'Check Connection' dialog box is open, displaying a green checkmark and the message: 'HTTP request (without authentication) to JouleOps_FastAPI destination succeeded.' The dialog has a 'Close' button. The background shows the 'Destination Details' for 'JouleOps_FastAPI', including 'Source: New destination from scratch', 'Type: HTTP', 'Description: (empty)', and 'Proxy Type: Internet'. The 'Authentication' section shows 'Authentication: NoAuthentication'. The bottom of the screen shows a Windows taskbar with the search bar and various application icons.

The screenshot displays the SAP BTP Cockpit interface, similar to the one above. The 'Destinations' table shows the same two entries. The 'JouleOps-MCP' destination is now selected, and its details are shown on the right. A 'Check Connection' dialog box is open, displaying a green checkmark and the message: 'HTTP request (without authentication) to JouleOps-MCP destination succeeded.' The dialog has a 'Close' button. The background shows the 'Destination Details' for 'JouleOps-MCP', including 'Source: New destination from scratch', 'Type: HTTP', 'Description: (empty)', and 'Proxy Type: Internet'. The 'Authentication' section shows 'Authentication: NoAuthentication'. The bottom of the screen shows a Windows taskbar with the search bar and various application icons, including a weather widget showing '25°C Partly sunny'.

12. Conclusion

JouleOps is a layered enterprise AI application that connects SAP HANA Cloud, FastAPI, MCP and SAP BTP connectivity. The project demonstrates how business data can be exposed through controlled operations instead of giving the AI direct database access.

The most important technical parts are the HANA database, the five FastAPI capabilities, the MCP tool layer, the ngrok HTTPS connection and the two BTP destinations.

The ngrok 404 problem was solved by making sure the tunnel forwarded to the same port where FastAPI was running: port 8000. The successful public /health response confirmed the connection.

The final Joule layer depends on the capabilities and permissions of the assigned SAP BTP tenant. If the tenant restricts live Joule UI access, the restriction should be stated clearly while the independently validated backend, MCP and BTP layers are submitted as evidence.